Année scolaire 2025 – 2026

Lycée Victor Hugo - Colomiers

Terminale NSI

AL
Algorithmie

Thomas Ricart
ricart.lvh@laposte.net

5 novembre 2025

AL01 – Algorithmes sur les Arbres binaires

1. Parcours sur les arbres binaires

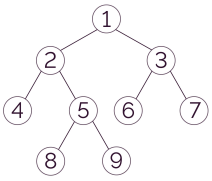
Définition 1: Parcours d'arbres binaires

Parcourir un arbre revient à ordonner la liste des noeuds et des feuilles de celui-ci dans un certain ordre. Il peut se faire:

1. en **profondeur d'abord**:
   - le parcours **préfixe**: on liste chaque sommet la première fois où on les rencontre;
   - le parcours **suffixe** (ou **postfixe**): on liste chaque sommet la dernière fois où on les rencontre;
   - le parcours **infixe**: on liste chaque sommet ayant un fils gauche la seconde fois où on le rencontre.
2. en **largeur d'abord**: c'est un parcours hiérarchique où on épuise chaque niveau de l'arbre avant de passer au suivant.

1.1. Parcours en profondeur d'abord

Parcours préfixe




```python
def prefixe(arbre):
    if not est_vide(arbre):
        traiter_racine(arbre)
        prefixe(sag)
        prefixe(sad)
```

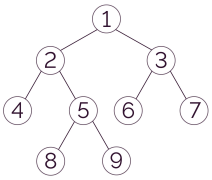
- la fonction `prefixe` prend pour paramètre un arbre binaire;
- si l'arbre n'est pas vide on traite l'étiquette de la racine de l'arbre;
- on parcourt en préfixe le SAG de l'arbre;
- on parcourt en préfixe le SAD de l'arbre.

Exercice 1: Parcours préfixe d'un arbre binaire

1. Donner la liste dans l'ordre des sommets parcourus en préfixe: .....
2. On stocke le nom des sommets dans une liste. Cela revient à faire: `[racine]` prefixe(sag) + prefixe(sad).
   Créer la trace pour l'arbre exemple:

Appel	Condition IF	Appel récursif	Etat de la liste de valeurs
`prefixe(1)`	True	`[1] + prefixe(2) + prefixe(3)`	`[1]`

Parcours infixe



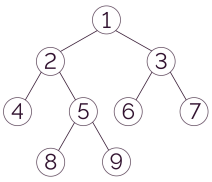

```python
def infixe(arbre):
    if not est_vide(arbre):
        infixe(sag)
        traiter_racine(arbre)
        infixe(sad)
```

- la fonction `infixe` prend pour paramètre un arbre binaire;
- si l'arbre n'est pas vide on parcourt le SAG de l'arbre en infixe;
- on traite l'étiquette de la racine;
- on parcourt en infixe le SAD de l'arbre.

Exercice 2: Parcours infixe d'un arbre binaire

1. Donner la liste dans l'ordre des sommets parcourus en infixe: .....
2. On stocke le nom des sommets dans une liste. Cela revient à faire: infixe(sag) + [racine] + infixe(sad).
   Créer la trace pour l'arbre exemple.

Parcours postfixe




```python
def postfixe(arbre):
    if not est_vide(arbre):
        postfixe(sag)
        postfixe(sad)
        traiter_racine(arbre)
```

- la fonction `postfixe` prend pour paramètre un arbre binaire;
- si l'arbre n'est pas vide on parcourt le SAG de l'arbre en postfixe;
- on parcourt en postfixe le SAD de l'arbre.
- on traite l'étiquette de la racine;

Exercice 3: Parcours postfixe d'un arbre binaire

1. Donner la liste dans l'ordre des sommets parcourus en infixe: .....
2. On stocke le nom des sommets dans une liste. Cela revient à faire: infixe(sag) + [racine] + infixe(sad).
   Créer la trace pour l'arbre exemple.

1.2. Parcours en largeur d'abord

Propriété 1:

L'idée du parcours en largeur d'abord est de s'occuper de chaque niveau de l'arbre en commençant par le niveau de la racine 1. A chaque niveau on lira les noeuds de la gauche vers la droite.

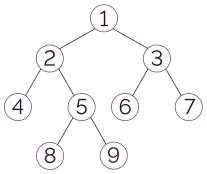
```
liste_vus <- liste vide                    // liste des sommets traités
F <- File()                                // file d'attente des sommets à traiter
enfiler(F, racine)                         // enfiler le noeud racine
tant que F non vide:                       // tant qu'il y a des sommets à traiter
    n <- defiler(F)                        // récupère le premier sommet à traiter
    liste_vus <- liste_vus + valeur(n)     // on ajoute n à la liste traitée
    si SAG(n) non vide:                    // enfiler le traitement du SAG
        enfiler(F, SAG(n))
    si SAD(n) non vide:                    // enfiler le traitement du SAD
        enfiler(F, SAD(n))
renvoyer liste_vus
```

Exercice 4: Parcours en largeur d'abord

1. Donner la liste dans l'ordre des sommets parcourus: .....
2. Créer une trace d'appels de ce type de parcours

while	n	F	liste_vus	si SAG	si SAD	F

2. Arbres binaires en python



Dans la suite des exercices, l'arbre par défaut qui sera considéré sera l'arbre ci-contre.

L'objectif de cet exercice est de pouvoir créer cet arbre à partir d'une classe Nœud et de pouvoir interagir avec cet arbre.

Spécification de la classe Nœud

- attibuts:
  - `valeur` (int);
  - `gauche` (Noeud): le nœud racine du sous arbre gauche;
  - `droite` (Noeud): le nœud racine du sous arbre droit.
- méthodes:
  - `get_valeur()` qui récupère sous forme d'entier la valeur du noeud;
  - `get_gauche()` qui récupère sous forme de Noeud la racine su SAG;
  - `get_droite()` qui récupère sous forme de Noeud la racine su SAD;
  - `set_valeur(valeur)` qui modifie la valeur du nœud;
  - `set_gauche(noeud)` qui modifie la racine du SAG;
  - `set_droite(noeud)` qui modifie la racine du SAD;
  - `__str__()` qui surcharge la commande `print` et affiche sa valeur;
  - `__repr__()` qui surcharge l'affichage de la valeur du nœud dans la console et affiche sa valeur.

Exercice 5: Création de la classe Noeud

Code Python

```python
# Création des instance de la classe Noeud
N1, N2, N3 = Noeud(1), Noeud(2), Noeud(3)
N4, N5, N6 = Noeud(4), Noeud(5), Noeud(6)
N7, N8, N9 = Noeud(7), Noeud(8), Noeud(9)

# Création des arêtes grâce aux setters.
# A compléter

assert (N1.get_valeur() == 1) and  (N8.get_valeur() == 8)
assert N1.get_gauche().get_valeur() == 2
assert N2.get_droite().get_gauche().get_valeur() == 8
```

Tester votre code avec les tests suivants:

```
>>> A = Noeud(1)
>>> print(A)
1
>>> A
1
```

Spécification de la classe ArbreBinaire

- attibuts:
  - `racine` (Noeud) le nœud racine;
- méthodes:
  - `get_racine()` qui renvoie le nœud racine;
  - `prefixe(noeud = None)` qui renvoie la liste des nœuds dans l'ordre de visite en préfixe. Appelle la méthode `prefixe_rec(noeud)`
  - `infixe(noeud = None)` qui renvoie la liste des noeuds dans l'ordre de visite en infixe. Appelle la méthode `infixe_rec(noeud)`
  - `postfixe(noeud = None)` qui renvoie la liste des noeuds dans l'ordre de visite en postfixe. Appelle la méthode `postfixe_rec(noeud)`

Exercice 6: Création de la classe ArbreBinaire

On pourra s'inspirer ce ce code pour les classes de parcours:

Code Python

```python
def prefixe(self, noeud = None):
    if noeud is None::
        ...
    if noeud is None:
        return None
    return self.prefixe_rec(...........)
def prefixe_rec(self, noeud):
    if ................. :
        return ...........
    else:
        return ...........
```

Code Python

```python
A = ArbreBinaire(N1)

assert A.prefixe()  == [1, 2, 4, 5, 8, 9, 3, 6, 7]
assert A.infixe()   == [4, 2, 8, 5, 9, 1, 6, 3, 7]
assert A.postfixe() == [4, 8, 9, 5, 2, 6, 7, 3, 1]

assert A.prefixe(N2) ==  [2, 4, 5, 8, 9]
assert A.infixe(N2) ==   [4, 2, 8, 5, 9]
assert A.postfixe(N2) == [4, 8, 9, 5, 2]
```

Exercice 7: Taille, hauteur et nombre de feuilles de l'arbre

1. Déterminer la relation de récurrence qui permet de calculer la taille d'un arbre. .....
2. Écrire une méthode `taille` qui renvoie la taille de l'arbre.
3. Déterminer la relation de récurrence qui permet de calculer la hauteur d'un arbre. .....
4. Écrire une méthode `hauteur` qui renvoie la hauteur de l'arbre.
5. Déterminer la relation de récurrence qui permet de calculer le nombre de feuilles d'un arbre. .....
6. Écrire une méthode `nb_feuilles` qui renvoie le nombre de feuilles de l'arbre.
7. Déterminer la relation de récurrence qui permet de chercher une valeur dans un arbre. .....
8. Écrire une méthode `recherche` qui renvoie `True` la valeur est présente.
9. Écrire la méthode `parcours_largeur` qui renvoie la liste des nœuds lus suivant le parcours en largeur.

3. Arbres binaires de recherche en Python

Dans toute la suite de l'exercice, nous allons pouvoir réutiliser la classe `Noeud` et `ArbreBinaire` créées précédemment.

Exercice 8: Algorithmes sur les ABR

1. Créer une méthode `insertion_abr` et `insertion_abr_rec` qui insère automatiquement une clé dans l'ABR.

Code Python
```python
def insertion_abr(self, cle):
    if self.racine is None:
        self.racine = ...
    else:
        return self.insertion_abr_rec(cle, self.racine)
def insertion_abr_rec(self, cle, noeud):
    if cle < ...
        if noeud.gauche is None:
            ...
        else:
            ...
    else:
        if noeud.droite is None:
            ...
        else:
            ...
```

2. Vérifier le bon fonctionnement de cette méthode en créant un arbre binaire en insérant les valeurs dans l'ordre suivant: `[5, 6, 3, 1, 2, 4, 7]`
3. Dessiner la structure de l'arbre implémenté.
4. Créer une méthode `afficher` et `afficher_rec` qui affiche l'arbre sous forme de liste de listes et vérifier que l'ABR est bien constitué:

Code Python
```python
def afficher_rec(self, noeud):
    if noeud is None:
        return None
    else:
        return [ ... ]
```

Code Python
```
>>> ABR.afficher()
[5, [3, [1, None , [2, None , None ]], [4, None , None ]], [6, None , [7, None , None ]]]
```

5. Créer une méthode `est_abr(self)` qui renvoie `True` si l'arbre est un ABR.
6. Créer une méthode `recherche_abr(self, valeur)` et `recherche_abr_rec` qui renvoie `True` si il existe un noeud valeur.

Code Python
```python
def recherche_abr_rec(self, val, noeud):
    if not noeud:
        return ...
    if noeud.valeur == val:
        return ...
    if noeud.valeur < val:
        return ...
    else:
        return ...
assert ABR.recherche_abr(3) == True
assert ABR.recherche_abr(9) == False
```

7. Implémenter sur le même principe une méthode `maximum` et `minimum` qui renvoie la valeur maximale et minimale présente dans l'ABR.

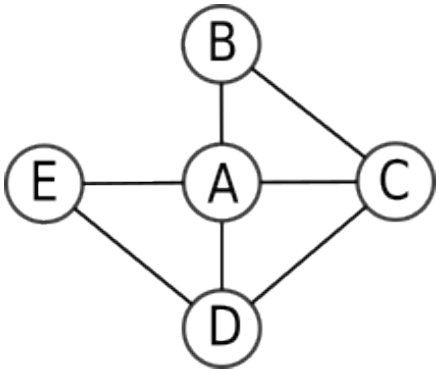
Code Python
```python
def minimum_abr(self, noeud = None):
    if noeud is None:
        noeud = self.racine
    return self.minimum_abr_rec(noeud)
def minimum_abr_rec(self, noeud):
    if noeud.valeur is None:
        return ...
    return ...
```

8. Vérifier le bon fonctionnement des méthodes `maximum` et `minimum`

AL02 – Algorithmes sur les graphes

1. Implémentation de la classe Graphe

Nous voulons créer le graphe ci-contre et qu'il puisse être créé grâce aux instructions suivantes:



Code Python

```python
g = Graphe(['A', 'B', 'C', 'D', 'E'])
g.ajoute_arete('A', 'B')
g.ajoute_arete('A', 'C')
g.ajoute_arete('A', 'D')
g.ajoute_arete('A', 'E')
g.ajoute_arete('B', 'C')
g.ajoute_arete('C', 'D')
g.ajoute_arete('D', 'E')
```

Nous souhaitons également tester si deux sommets sont voisins:

Code Python

```python
>>> g.sont_voisins('E', 'A')
True
>>> g.sont_voisins('E', 'B')
```

Enfin, nous voulons pouvoir obtenir facilement la liste de tous les voisins d'un sommet avec la méthode voisins:

Code Python

```python
>>> g.voisins('C')
['A', 'B', 'D']
```

Voici les spécifications de la classe `Graphe`:

```
class Graphe
********************
liste_sommets: (list) liste des sommets
adjacents: (dict) chaque sommet se voit attribuer une liste vide []: {sommet: []}
********************
ajoute_arete(Graphe, sommetA, sommetB)
sommet_existe(Graphe, sommet) -> bool
voisins(Graphe, sommet) -> list
sont_voisins(Graphe, sommetA, sommetB) -> bool
```

Exercice 1: Implémenter la classe `Graphe` en python

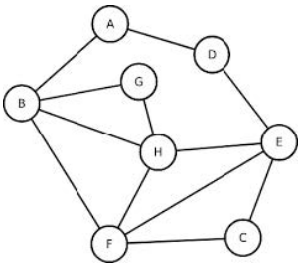
2. Parcours de Graphes

Définition 1: Parcours en largeur

Parcourir un graphe en largeur revient à ordonner la liste des sommet en largeur d'abord:

L'algorithme de parcours en largeur (ou BFS, pour Breadth First Search en anglais) permet le parcours d'un graphe ou d'un arbre de la manière suivante:

- on commence par explorer un sommet source
- puis ses successeurs non explorés
- puis les successeurs non explorés des successeurs
- etc.



L'algorithme de parcours en largeur permet de calculer les distances de tous les sommets depuis un sommet source dans un graphe non-pondéré (orienté ou non-orienté). Il peut aussi servir à déterminer si un graphe non-orienté est connexe.
Cet algorithme diffère de l'algorithme de parcours en profondeur par le fait que, à partir d'un sommet source S, il liste d'abord les voisins de S pour ensuite les explorer un par un. Ce mode de fonctionnement utilise donc une file dans laquelle il prend le premier sommet et place en dernier ses voisins non encore explorés.

Les sommets déjà visités sont marqués afin d'éviter qu'un même sommet soit exploré plusieurs fois. Dans le cas particulier d'un arbre, le marquage n'est pas nécessaire.

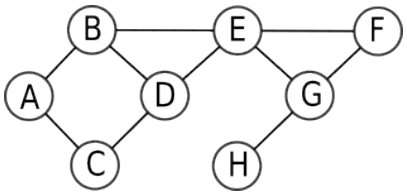
```
vus est une liste
en_attente est une liste
ajouter depart à en_attente
Tant que en_attente n'est pas vide:
    sommet <- défiler en_attente
    voisins <- liste des voisins de sommet ordonnés croissants
    Pour chaque voisin de voisins:
        ajouter à en_attente les voisins non traites et non déjà en attente
    ajouter sommet à vus
renvoyer vus
```

Exercice 2: Parcours en largeur BFS

1. On considère le graphe suivant. Ajouter les arêtes grâce à la méthode `ajoute_arete`.

Code Python

```python
g = Graphe(['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'])
```



2. Donner le parcours en largeur BFS depuis B .....
3. Donner le parcours en largeur BFS depuis D .....
4. Donner le parcours en largeur BFS depuis G .....

Exercice 3: Donner la trace du parcours en largeur d'abord du graphe l'exercice 2 depuis A.

vus	en_attente	while	sommet	voisins	en_attente

Exercice 4: Implémenter le parcours BFS

Code Python

```python
def BFS(self, depart):
    vus = []
    en_attente = [depart]
    while ....
        sommet = ....
        voisins = ....
        for ....
            ....
        ....
        ....
        ....
```

Définition 2: Application du BFS: recherche du plus court chemin

L'algorithme BFS découvre les sommets par cercles concentriques autour du point de départ. On découvre d'abord tous les sommets à la distance 1 du point de départ, puis à la distance 2 etc. Un sommet situé à la distance 5 sera découvert en tant que voisin d'un sommet à la distance 4...
On comprend donc que si on arrive à se souvenir du sommet parent de chaque sommet (celui qui lui a permis d'être découvert), on pourra alors reconstituer un chemin permettant de remonter au point de départ.
Nous allons pour cela nous servir d'une structure de dictionnaire pour associer à chaque sommet son sommet-parent.
Il faudra ensuite une fonction pour recréer le chemin.

```
recherche_chemin(graphe, depart, arrivee):
    vus, en_attente est une liste
    ajouter depart à en_attente
    parent est un dictionnaire
    Tant que en_attente n'est pas vide:
        sommet <- défiler en_attente
        voisins <- liste des voisins de sommet
        Pour chaque voisin de voisins:
            ajouter à en_attente les voisins non traites
                      et non déjà en attente
            crée entrée de dictionnaire {voisin : sommet}
            si le voisin est l'arrivée:
                reconstituer le chemin (depart, arrivee, parent)
        ajouter sommet à vus
    renvoyer "non trouvé"
```

```
reconstitue_chemin(depart, arrivee, parent):
    sommet <- arrivee
    chemin <- arrivee
    Tant que le sommet n'est pas celui de départ:
        sommet <- parent[sommet]
        chemin <- sommet + chemin
    renvoyer chemin
```

Exercice 5: Implémenter les deux fonctions et tester votre programme sur le graphe de l'exercice 2

1. entre A et G .....
2. entre H et C .....
3. entre B et G .....

Définition 3: Parcours en profondeur

Le parcours en profondeur (DFS: Depth Firest Search) est un parcours qui va aller le plus loin possible sans se préoccuper des autres voisins non visités: on va visiter le premier de ses voisins non traités etc. Lorsqu'il n'y a plus de voisins, on revient en arrière pour visiter le dernier voisin non visité.

```
vus est une liste
en_attente est une liste
ajouter depart à en_attente
Tant que en_attente n'est pas vide:
    sommet <- dépiler en_attente
    si le sommet n'est pas déjà vu:
        voisins <- liste des voisins de sommet ordonnés décroissant
        Pour chaque voisin de voisins:
            si le voisin n'est pas déjà vu:
                ajouter à en_attente les voisins non traites et non déjà en attente
        ajouter sommet à vus
renvoyer vus
```

Exercice 6: Implémenter le parcours DFS en python

Code Python

```python
def DFS(self, sommet_depart):
    ...
```

Exercice 7: Proposer une version récursive du parcours dfs

Code Python

```python
def dfs2(self, sommet_depart):
    vus = []
    en_attente = [sommet_depart]
    return self.dfs2_rec(vus, en_attente)
def dfs2_rec(self, vus, en_attente):
    if en_attente == []:
        return vus
    else:
        sommet = en_attente.pop()
        if sommet not in vus:
            ....
            voisins = sorted(self.voisins(sommet), reverse = False)
            for v in voisins:
            ....
            ....
            ....
        return self.dfs2_rec(vus, en_attente)
```

AL03 – Diviser pour régner

1. L'algorithme de dichotomie

1.1. La version impérative

Exercice 1: Dichotomie impérative

Ecrire la fonction `recherche_dicho_imperative(tab, val)` qui renvoie `True` si `val` est dans `tab` et `False` sinon.

Code Python

```python
def recherche_dicho_imperative(tab, val):
    pass

tab = [1, 5, 7, 9, 12, 13]
assert recherche_dichotomique(tab, 12) == True
assert recherche_dichotomique(tab, 17) == False
```

À chaque tour de la boucle while, la taille de la liste est divisée par 2. Ceci confère à cet algorithme une complexité logarithmique (bien meilleure qu'une complexité linéaire).

1.2. Version récursive

Remarque 1: Le slicing permet de ne garder qu'un bout de liste

Code Python

```python
L = [3, 2, 5, 6, 9, 8, 5]
assert L[:] == L
assert L[2:] == [5, 6, 9, 8, 5]
assert L[:4] == [3, 2, 5, 6]
assert L[2:4] == [5, 6]
```

Exercice 2: Écrire la fonction `dichotomie_rec` avec slicing et la tester

2. Diviser pour régner

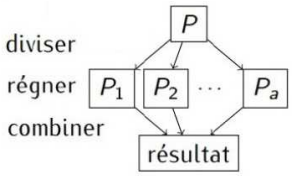
Définition 1: Diviser pour régner

Un problème peut se résoudre en employant le paradigme **Diviser pour Régner** lorsque:

- il est possible de décomposer ce problème en sous problèmes indépendants;
- la taille de ces sous problèmes est une fraction du problème final.

La méthode est composée de trois phases:

- la `division`: le problème initial est divisé en sous problème de taille inférieure;
- le `règne`: chaque sous-problème est résolu soit récursivement, soit directement si trivial;
- la `combinaison`: les solutions des sous problèmes sont combinés pour obtenir la solution du problème initial.



P est un problème de taille n

$P_1, P_2, \ldots, P_a$ sont des problèmes de taille $\frac{n}{a}$

Approche récursive : chaque P_i est à son tour décomposé

Exercice 3: Algorithme de recherche dans une liste non triée

```
si la liste est vide alors l'élément n'appartient pas à la liste
si la liste ne comporte qu'un seul élément alors l'élément est présent s'il est égal à L[0]
sinon on recherche l'élément dans la liste gauche ou dans la liste droite
```

Écrire cette fonction `recherche(L, val)` en python

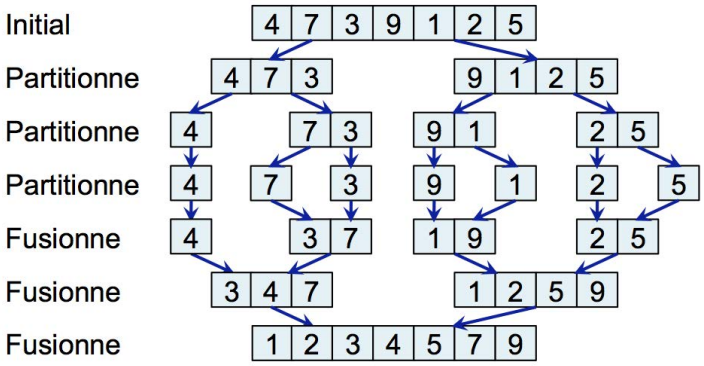
3. Le tri fusion

Pour trier une paquet de copie dans l'ordre alphabétique, il est possible de scinder le paquet en deux sous-paquets de même taille puis de trier chacun de ces sous-paquets avant de les fusionner pour obtenir le paquet trié.

Pour trier ce sous-paquet, on peut utiliser la même technique...

On peut scinder les sous-paquets jusqu'à avoir une certitude: si le paquet n'a qu'une seule copie alors ce paquet est trié.

```
si la liste ne contient qu'un seul élément:
    la liste est triée
sinon:
    on fusionne les deux sous listes triées
```



Exercice 4: Écrire la fonction `division(L)` qui renvoie un tuple contenant les deux sous-listes

Code Python

```python
def division(L:list)->tuple:

L1 = [1, 4, 6, 15]
L2 = [3, 9, 10]
assert division (L1) == ([1, 4], [6, 15])
assert division (L2) == ([3], [9, 10])
assert division ([1]) == ([] , [1])
```

Exercice 5: Écrire la fonction `fusion(L1, L2)` qui renvoie une liste fusionnée

```
fonction fusion(L1, L2, L)
    si L1 OU L2 est vide:
        on renvoie L1 + L2
    si le premier élément de L1 et plus petit que le premier élément de L2:
        on insère le premier élément de L1 dans la liste fusionnée
    si le premier élément de L2 et plus petit que le premier élément de L1:
        on insère le premier élément de L2 dans la liste fusionnée
    On appelle récursivement fusion avec le nouveau L1, le nouveau L2 et le nouveau L
```

Code Python

```python
def fusion(L1, L2, L = None):
    if L = None:
        L = []
    return fusion_rec(L1, L2, L)
def fusion_rec(L1, L2, L):
    if L1 == [] or L2 == []:
        return ....
    if ....... < .........:
        ......................
    else:
        ......................
    L = ......................
    return fusion_rec(........, ........, ..........)

assert fusion(L1, L2) == [1, 3, 4, 6, 9, 10, 15]
```

Exercice 6: Ecrire alors la fonction récursive `tri_fusion(L1, L2)`

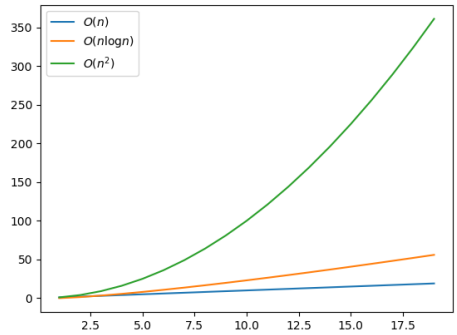
Code Python

```python
def tri_fusion(L:list)->list:

L = [5, 7, 2, 1, 9, 8, 6]
assert tri_fusion(L) == [1, 2, 5, 6, 7, 8, 9]
```

Propriété 1: Complexité de l'algorithme

L'instruction finale
`fusion(tri_fusion(L1), tri_fusion(L2))`
lance deux appels à la fonction `tri_fusion` (avec certes des données d'entrée deux fois plus petites).
On peut alors montrer que la complexité du tri fusion est en `O(n log n)`



Exercice 7: Dessiner de manière schématisé le tri fusion de `[4, 8, 3, 1, 7, 6]`

Exercice 8: Ecrire la trace de cet algorithme lors de l'appel de `tri_fusion([4, 8, 3, 1, 7, 6])`

Appel	n	IF	L1	L2	appel récursif fusion	valeur
trifusion([4,8,3,1,7,6])	6	F	[4,8,3]	[1,7,6]	trifusion([4,8,3]), trifusion([1,7,6])	

Exercice 9: Application à la recherche de maximum dans une liste

```
si la liste ne contient qu'un seul élément:
    c'est le plus grand
on coupe la liste en deux
on récupère le plus grand de la liste de gauche
on récupère le plus grand de la liste de droite
on renvoie le plus grand des deux
```

On pourra utiliser la fonction `max` pour trouver le maximum entre deux valeurs.

Exercice 10:
Application au calcul de l'exponentiation rapide
Il est facile de vérifier que:

- $a^0 = 1$
- si n est pair, $a^n = (a \times a)^{\frac{n}{2}}$
- si n est impair: $a^n = a \times (a \times a)^{\frac{n-1}{2}}$

AL04 – Programmation dynamique

1. Retour sur la suite de Fibonacci

Il est possible de coder de manière récursive les termes de la suite de Fibonacci:

Code Python

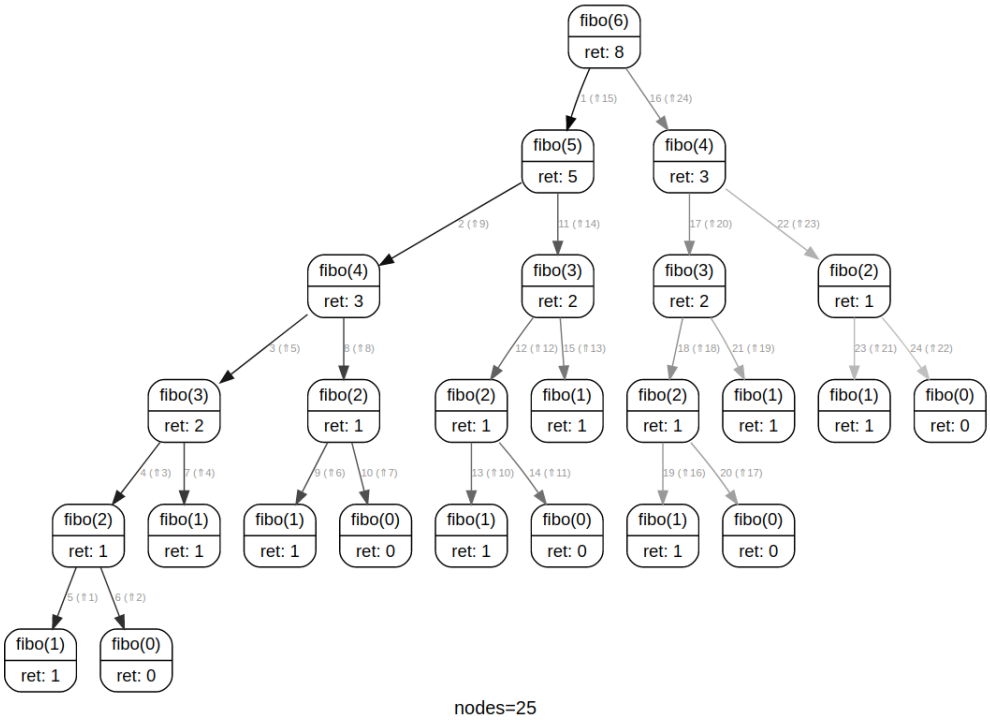
```python
def fibo_rec(n):
    if n == 0 or n == 1:
        return n
    return fibo_rec(n-1) + fibo_rec(n-2)
```

il est possible de mesurer le temps de calcul de `fibo(40)`:

Code Python

```python
import time
t0 = time.time()
fibo(40)
print(time.time() - t0)
```

La cause est la multitude d'appels récursifs qui nous conduit à refaire des calculs déjà effectués. On peut le voir sur l'arbre d'appels de `fibo(6)`:



Le calcul de `fibo(2)` se retrouve 5 fois dans l'arbre. Il est donc possible de pouvoir optimiser cet algorithme.

2. Programmation Top - Bottom

Se souvenir des calculs déjà effectués

On peut se souvenir des résultats par un phénomène de **mémorisation**.
L'idée est de stocker le résultats de chaque calcul, par exemple dans un dictionnaire.

- soit le calcul a déjà été effectué: on a donc juste à le lire dans le dictionnaire;
- soit le calcul n'a jamais été effectué: on l'effectue et on le stocke dans le dictionnaire.

Code Python

```python
def fibonacci_memo(n, memo={0: 0, 1: 1}):
    if n not in memo:
        memo[n] = fibonacci_memo(n - 1, memo) + fibonacci_memo(n - 2, memo)
    return memo[n]
```

Propriété 1: Approche Top - Down: Mémorisation

C'est une approche récursive où on commence par résoudre le problème principal en appelant des sous-problèmes, et on mémorise les résultats pour éviter de les recalculer.

- Utilise la récursion.
- Stocke les résultats intermédiaires dans un cache (dictionnaire par exemple).
- Peut souffrir d'un trop grand nombre d'appels récursifs.

Le principe est donc:

- On commence par `fibonacci(6)`, qui appelle `fibonacci(5)` et `fibonacci(4)`, etc.
- Dès qu'un calcul est fait, il est stocké pour éviter de le refaire.
- L'appel récursif descend jusqu'à `fibonacci(0)` et `fibonacci(1)`, puis remonte en réutilisant les valeurs mémorisées.

Exercice 1: Comparer le temps d'exécution de la méthode Top Bottom

3. Programmation Bottom - Up

Définition 1: Programmation Bottom - Up

C'est une approche itérative où on commence par résoudre les plus petits sous-problèmes en premier, puis on construit la solution du problème global à partir de ceux-ci.

- Utilise une boucle au lieu de la récursion.
- Évite la surcharge de la pile d'appels.
- Généralement plus efficace en termes de mémoire et de performance.

Le principe est donc:

- On commence par remplir un tableau en partant de `fib[0]` et `fib[1]`, puis on calcule tous les termes jusqu'à `fib[n]`.
- On évite la récursion, donc pas de problème de dépassement de pile.

Exercice 2: Fibonacci en Bottom - Up. Compléter et comparer le temps d'exécution

Code Python

```python
def fibonacci_bottom_up(n):
    dict_fibo = {0:0, 1:1}
    for k in range(..... , ......):
        dict_fibo[k] = ..............................
    return ....................................
```

4. Méthode sans mémorisation

Pour calculer les valeurs de la suite de Fibonacci il suffit finalement de connaitre les deux précdentes. Il suffit donc de les sauvegarder dans une liste par exemple.

Code Python

```python
def fibonacci_simple(n):
    if n <= 1:
        return n
    return fibonacci_simple_rec(n, 1, [0, 1])
def fibonacci_simple_rec(n, k, liste):
    if k == n:
        return ..................
    return fibonacci_simple_rec(..............................
```

5. Bilan

Propriété 2: Bilan programmation dynamique

Lors d'un calcul effectué de manière récursive, il peut arriver que de multiples appels récursifs soient identiques. Pour éviter de recalculer plusieurs fois la même chose, on peut stocker les résultats intermédiaires. On appelle cette technique la mémoriation. Cette technique minimise le nombre d'opérations et accélère grandement l'exécution du programme. Le prix à payer est l'utilisation d'une structure de stockage des valeurs intermédiaires, et donc une augmentation de la mémoire utilisée par le programme.

Lors d'un calcul effectué de manière itérative, il est parfois plus simple de commencer par une petite version du problème pour progressivement remonter vers la solution du problème global.

AL05 – Recherche textuelle

Définition 1: Recherche textuelle

La recherche d'une sous-chaine de caractères dans une plus grande chaine de caractère est un grand classique en algorithmique. Ces algorithmes sont notamment utilisés en bioinformatique à propos de la recherche de séquence ADN. L'ADN est une molécule en double hélices qui est constituée d'un enchainement de quatre nucléotides différents : A, C, G et T pour Adénine, Cytosine, Guanine et Thymine. Le séquençage de l'ADN consiste à obtenir la suite des nucléotides et à y rechercher des motifs correspondant à différents gènes. L'optimisation des algorithmes de recherche est un enjeu majeur pour la recherche génétique.

- La chaine de caractère recherchée est appelée le motif ou la clé.
- La chaine de caractère dans laquelle s'effectue la recherche est appelée la séquence ou le texte.
- Chaque apparition du motif dans le texte est appelée une occurrence.

Exercice 1: La méthode `find` de Python

`activite`

1. Recherche textuelle naive

Définition 2: Recherche naive

- Regarder les 5 premières minutes de la vidéo:
  `https://grafikart.fr/tutoriels/boyer-moore-horspool-1197`
- Observer l'animation suivante: `https://boyer-moore.codekodo.net/recherche_naive.php`

L'approche naive pour la recherche textuelle est d'effectuer une comparaison du motif avec la séquence en commençant par la gauche et en comparant les caractères un par un de gauche à droite puis en décalant le motif vers la droite caractère par caractère. On parle ici de force brute.

Exercice 2: Exercice sur la recherche textuelle naive

`url`

2. Le principe de l'algorithme Boyer Moor Horspool

Définition 3: Algorithme de recherche naive en partant à l'envers

Observer l'animation suivante: `https://boyer-moore.codekodo.net/recherche_boyer.php`

L'**algorithme de Boyer-Moore**, optimise l'algorithme naif en diminuant le nombre de comparaisons par l'élimination de celles qui sont inutiles.
Les trois grandes idées de l'algorithme de Boyer-Moore sont:

- La comparaison de droite à gauche des caractères du motif et du texte.
- Le décalage du motif de plusieurs crans vers la droite lorsque c'est possible plutôt que de le faire glisser d'un seul caractère à la fois.
- Le pré-traitement du motif afin d'en avoir une bonne connaissance et permettre les grands décalages du motif.

Exercice 3: Algorithme de Boyer Moore

`url`